# C Programming Language — Beginner's Guide

C PROGRAMMING LANGUAGE

A Beginner's Guide

1. WHAT IS C?

C is a general-purpose programming language developed by Dennis Ritchie at Bell Labs in the early 1970s. It is widely used in operating systems, embedded devices, libraries, and performance-sensitive software.

C programs are commonly compiled into machine code. The language offers direct control over memory, but programmers must take care to avoid invalid memory access and other errors. This guide uses examples suitable for C17.

2. YOUR FIRST PROGRAM

```
#include <stdio.h>

int main(void) {

printf("Hello, world!\n");

return 0;

}
```

• #include <stdio.h> provides declarations for standard input/output functions.

• main is the entry point of a hosted C program.

• printf writes formatted output. \n starts a new line.

• return 0 indicates successful completion.

• Statements usually end with a semicolon; braces group statements.

Save the code as hello.c. With GCC installed, compile and run it:

```
gcc -std=c17 -Wall -Wextra -Wpedantic hello.c -o hello

./hello
```

On Windows, the executable may be hello.exe; in PowerShell, run .\hello.exe.

3. VARIABLES AND DATA TYPES

A variable is a named object that stores a value. Declare its type before using it, and initialize it before reading its value.

```
int age = 20;

float price = 19.5f;

double distance = 123.456;

char grade = 'A';

const int limit = 100;
```

Common types:

• char: stores a character-sized integer value.

• int: stores integers.

• float and double: store floating-point values.

• void: represents the absence of a value in contexts such as function return types.

• _Bool: stores 0 or 1; C17's <stdbool.h> provides bool, true, and false.

Type sizes depend on the implementation. sizeof(char) is always 1, but a C byte is not necessarily eight bits. Use sizeof to inspect object sizes; its result has type size_t.

```
printf("int uses %zu bytes\n", sizeof(int));
```

Comments:

```
// A single-line comment

/* A comment that can span multiple lines */
```

4. INPUT AND OUTPUT

printf uses format specifiers:

• %d: int

• %u: unsigned int

• %f: double (float arguments are promoted to double)

• %c: character

• %s: null-terminated string

• %zu: size_t

Simple integer input:

```
int number;
printf("Enter an integer: ");
if (scanf("%d", &number) == 1) {
printf("You entered %d\n", number);
} else {
printf("Invalid input\n");
}
```

The & operator obtains the address where scanf stores the result. Always check the return value. For robust input handling, prefer fgets followed by strtol, with error and range checks. scanf's %f expects float*, whereas %lf expects double*.

## 5. OPERATORS

Arithmetic: +, -, *, /, %

Comparison: ==, !=, <, >, <=, >=

Logical: && (AND), || (OR), ! (NOT)

Assignment: =, +=, -=, *=, /=, %=

Increment and decrement: ++, --

Bitwise: &, |, ^, ~, <<, >>

Examples:

```
int a = 7, b = 2;
int quotient = a / b; // 3: integer division
int remainder = a % b; // 1
double ratio = (double)a / b; // 3.5
```

Use == to compare and = to assign. Division by zero is invalid. Signed integer overflow has undefined behavior. Parentheses make complex expressions easier to read.

## 6. DECISIONS

if / else:

```
if (age >= 18) {
printf("Adult\n");
```

```
} else {
printf("Under 18\n");
}
```

switch:

```
switch (grade) {
case 'A':
printf("Excellent\n");
break;
case 'B':
printf("Good\n");
break;
default:
printf("Keep practicing\n");
}
```

Without break, execution may continue into the following case. In conditions, zero is false and nonzero is true.

7. LOOPS

for: useful when the iteration count is known.

```
for (int i = 0; i < 5; ++i) {
printf("%d\n", i);
}
```

while: tests its condition before each iteration.

```
int count = 3;
while (count > 0) {
printf("%d\n", count);
--count;
}
```

do / while: executes its body at least once.

```
int value = 0;
do {
++value;
} while (value < 3);
```

break exits the nearest loop or switch. continue skips to the next loop iteration.

## 8. FUNCTIONS

Functions organize reusable operations. Declare a function before calling it.

```
#include <stdio.h>

int add(int a, int b);

int main(void) {
printf("%d\n", add(3, 4));
return 0;
}

int add(int a, int b) {
return a + b;
}
```

C passes arguments by value: each parameter receives a copy. To modify a caller's object, pass its address and access it through a pointer. Local automatic variables ordinarily exist only while their block is executing.

## 9. ARRAYS

An array stores a fixed number of elements of the same type.

```
int marks[4] = {75, 80, 92, 68};
int total = 0;
for (int i = 0; i < 4; ++i) {
total += marks[i];
}
```

Array indices start at 0. For marks, valid indices are 0 through 3. C does not automatically check bounds; accessing outside an array has undefined behavior.

For an actual array in its declaration scope:

size_t count = sizeof(marks) / sizeof(marks[0]);

This technique does not determine array length from a pointer. Array parameters in functions are adjusted to pointer parameters, so pass the element count separately.

10. STRINGS

A C string is a sequence of characters terminated by '\0'.

char greeting[] = "Hello"; // Six elements including '\0'

printf("%s\n", greeting);

Useful functions from <string.h>:

• strlen(s): length excluding the null terminator.

• strcmp(a, b): zero when strings are equal; negative or positive according to ordering.

• memcpy(dst, src, n): copies n bytes; the regions must not overlap.

Read a bounded line:

char name[80];

if (fgets(name, sizeof(name), stdin) != NULL) {

printf("Hello, %s", name);

}

fgets may retain the newline and may read only part of a long line. Ensure every string buffer has room for its terminator. Never use gets. Do not modify a string literal; use const char *message = "Hello" when referring to one.

11. POINTERS

A pointer stores an address.

int score = 10;

int *p = &score;

*p = 25; // Changes score to 25

& obtains an address; * dereferences a pointer.

Example of modifying a caller's variable:

```
void increment(int *value) {
if (value != NULL) {
++(*value);
}
}
// Call with: increment(&score);
```

A pointer must refer to a valid, suitably typed object before dereferencing. Do not dereference NULL, uninitialized pointers, or pointers to objects whose lifetime has ended. Pointer arithmetic is restricted to an array and its one-past position; the one-past pointer must not be dereferenced.

12. DYNAMIC MEMORY

Use <stdlib.h> for malloc, calloc, realloc, and free.

```
#include <stdio.h>
#include <stdlib.h>

int main(void) {
size_t count = 5;
int *values = malloc(count * sizeof *values);
if (values == NULL) {
fprintf(stderr, "Allocation failed\n");
return EXIT_FAILURE;
}
for (size_t i = 0; i < count; ++i) {
values[i] = (int)i;
}
free(values);
values = NULL;
return EXIT_SUCCESS;
```

}

malloc returns uninitialized storage. calloc initializes allocated storage to all-bits-zero. Free each allocation when it is no longer needed, exactly once. Do not access freed memory. When allocation sizes come from input, check multiplication for overflow. Use a temporary pointer with realloc so that failure does not lose the original allocation.

13. STRUCTURES, ENUMS, AND TYPEDEFS

A structure groups related values:

struct Student {

int id;

char name[40];

double average;

};

struct Student student = {1, "Asha", 88.5};

printf("%s: %.1f\n", student.name, student.average);

Use . for a structure object and -> for a pointer to a structure:

struct Student *ptr = &student;

printf("%d\n", ptr->id);

enum Color { RED, GREEN, BLUE };

enum Color selected = GREEN;

typedef unsigned long Counter;

Counter visits = 0;

typedef introduces an alternative name for a type; it does not create a distinct type.

14. FILE HANDLING

Use FILE pointers and functions from <stdio.h>.

FILE *file = fopen("notes.txt", "w");

if (file == NULL) {

perror("notes.txt");

```
return 1;
}
if (fprintf(file, "Learning C\n") < 0) {
fprintf(stderr, "Write failed\n");
}
if (fclose(file) == EOF) {
fprintf(stderr, "Closing file failed\n");
}
```

Common modes: "r" reads an existing file; "w" creates or truncates a file; "a" appends. Add "b" for binary mode where needed. Use fgets for text lines and fread/fwrite for blocks of data. Check operation results. When using fgetc, store its result in int so that EOF remains distinguishable from character values.

## 15. HEADERS AND THE PREPROCESSOR

#include includes header contents during preprocessing.

#define defines a macro.

#if, #ifdef, and #ifndef conditionally include code.

Example header with an include guard:

```
#ifndef MATH_UTILS_H
#define MATH_UTILS_H
int add(int a, int b);
#endif
```

Keep public declarations in headers and function definitions in .c files. Prefer functions over function-like macros where possible, because macros can evaluate arguments more than once.

Compile multiple source files:

```
gcc -std=c17 -Wall -Wextra -Wpedantic main.c math_utils.c -o app
```

## 16. COMPLETE PRACTICE PROGRAM: ARRAY STATISTICS

```
#include <stdio.h>
```

```
int main(void) {
int numbers[] = {12, 7, 19, 4, 10};
size_t count = sizeof(numbers) / sizeof(numbers[0]);
int sum = 0;
int largest = numbers[0];

for (size_t i = 0; i < count; ++i) {
sum += numbers[i];
if (numbers[i] > largest) {
largest = numbers[i];
}
}

printf("Sum: %d\n", sum);
printf("Average: %.2f\n", (double)sum / count);
printf("Largest: %d\n", largest);
return 0;
}
```

Expected output:

```
Sum: 52
Average: 10.40
Largest: 19
```

## 17. COMMON MISTAKES AND DEBUGGING

- Reading an uninitialized variable.
- Using = when == was intended.
- Accessing an array outside its bounds.
- Forgetting the null terminator in a string buffer.
- Using an incorrect printf or scanf format.
- Returning the address of a local automatic variable.
- Leaking allocations or freeing memory twice.

• Ignoring errors from input, file, or memory functions.

Enable compiler warnings and investigate them. Use a debugger such as GDB or LLDB to step through code. Where supported, build with runtime sanitizers:

gcc -std=c17 -g -Wall -Wextra -fsanitize=address,undefined hello.c -o hello

## 18. PRACTICE EXERCISES

1. Print your name and three lines of text.

2. Read two integers and display their sum, checking input success.

3. Determine whether an integer is even or odd.

4. Print a multiplication table using a loop.

5. Find the smallest element in an array.

6. Count vowels in a line read with fgets.

7. Write a function that swaps two integers using pointers.

8. Store and display three students using an array of structures.

9. Read a text file and count newline characters.

10. Allocate an integer array dynamically, fill it, and free it.

## RECOMMENDED NEXT STEPS

Practice one topic at a time. Then study recursion, linked lists, separate compilation, testing, and defensive input validation.

References for further study: The C Programming Language by Brian Kernighan and Dennis Ritchie (a classic covering an older version of C); C Programming: A Modern Approach by K. N. King; and the C reference at https://en.cppreference.com/w/c.

Note: Except for sections explicitly showing complete programs, examples are snippets to place within an appropriate program with the necessary standard headers.